

Top 50 Interview Questions on Data Types, ES6, OOPs & Async JS

Part 1: Data Types & Basics (12 Questions)

1. What are the primitive data types in JavaScript, and how do they differ from non-primitive (reference) types?
2. What is the difference between `null` and `undefined` ?
3. How does Type Coercion work in JavaScript? Explain the difference between `==` and `===` .
4. What is `NaN` ? Why does `NaN === NaN` return `false` ?
5. Explain the concept of “pass by value” vs. “pass by reference.”
6. How do you check if a variable is an Array?
7. What is the output of `typeof null` and why is it considered a bug?
8. What is the difference between a “shallow copy” and a “deep copy”?
9. What is a “Symbol” and what is a common use case for it?
10. Explain the concept of “hoisting” for variables and functions.
11. What are “truthy” and “falsy” values?
12. Why are strings immutable in JavaScript?

Part 2: Modern JavaScript (ES6+) (13 Questions)

1. What are the differences between `var` , `let` , and `const` regarding scope and hoisting?
2. Explain the “Temporal Dead Zone” (TDZ).
3. How does the `this` keyword behave differently in Arrow Functions compared to regular functions?
4. What is Object/Array Destructuring and how does it simplify code?
5. What is the difference between the Spread operator (`...`) and the Rest parameter (`...`)?
6. How do Template Literals differ from traditional string concatenation?
7. What are Default Parameters?
8. Explain the purpose of `Map` and `Set` . How do they differ from Objects and Arrays?
9. What are `WeakMap` and `WeakSet`?
10. What are JavaScript Modules (Import/Export) and why are they useful?
11. What is a Generator function (`function*`)?
12. What is “Optional Chaining” (`?.`)?
13. What is the “Nullish Coalescing” operator (`??`)?

Part 3: Object-Oriented Programming (OOP) (12 Questions)

1. Explain Prototypal Inheritance in JavaScript.
2. What is the **prototype** chain?
3. What is the difference between a Class (ES6) and a Constructor Function (ES5)?
4. What does the **new** keyword actually do under the hood?
5. Explain the **super** keyword in the context of class inheritance.
6. How do **call()** , **apply()** , and **bind()** differ?
7. What is a “closure” and how does it relate to data encapsulation?
8. How can you create “private” properties in a JavaScript class (using **#** syntax or closures)?
9. What is the difference between **__proto__** and **prototype** ?
10. What are “static” methods in a class?
11. What is “Method Overriding” in JavaScript?
12. How does **Object.create()** differ from **new Object()** ?

Part 4: Asynchronous JavaScript (13 Questions)

1. Explain the Event Loop. What is the Call Stack and the Callback Queue?
2. What is the difference between Synchronous and Asynchronous programming?
3. What are the three states of a Promise?
4. What is “Callback Hell” and how do Promises solve it?
5. Explain the difference between **Promise.all()** , **Promise.race()** , and **Promise.allSettled()** .
6. How does **async/await** work? Is it just syntactic sugar for Promises?
7. What is a Microtask vs. a Macrotask? (e.g., Promise vs. **setTimeout**).
8. How do you handle errors in **async/await** functions?
9. What happens if you **await** a non-promise value?
10. Why shouldn't you use **async** functions inside a **forEach** loop if you want them to run sequentially?
11. What is the difference between **setImmediate** (Node.js) and **setTimeout** ?
12. What is “Debouncing” vs. “Throttling”? (Often asked in the context of async events).
13. If a Promise is rejected but has no **.catch()** handler, what happens to the application?